Rails 3 and 4: More Special Cases

In Rails 1, attribute methods were implemented with a few dozen lines of code. In Rails 2, they had their own file and hundreds of lines of code. In Rails 3, they spanned nine files of source code, not including tests.

As Rails applications became larger and more sophisticated, the authors of the framework kept uncovering small twists, performance optimizations, and corner cases related to attribute methods. The code and the number of metaprogramming tricks it used grew with the number of corner cases. I'll show you only one of those corner cases, but even this single example is too long to fit in this chapter, so I will just show you a few snippets of code as quickly as I can. Brace yourself.

The example I picked is one of the most extreme optimizations in modern Rails. We've seen that Rails 2 improved performance by turning *Ghost Methods (57)* into *Dynamic Methods (51)*. Rails 4 goes one step further: when it defines an attribute accessor, it also turns it into an `UnboundMethod` and stores it in a method cache. If a second class has an attribute by the same name, and hence needs the same accessor, Rails 4 just retrieves the previously defined accessor from the cache and binds it to the second class. This way, if different attributes in separate classes happen to have the same name, then Rails defines only a single set of accessor methods and reuses those methods for all attributes. (I'm as surprised as you are that this optimization has a visible effect on performance—but in the case of Rails, it does.)

I'll start with code from deep inside the attribute methods implementation:

gems/activerecord-4.1.0/lib/active_record/attribute_methods/read.rb

```
module ActiveRecord
  module AttributeMethods
    module Read
      extend ActiveSupport::Concern

      module ClassMethods
        if Module.methods_transplantable?
          def define_method_attribute(name)
            method = ReaderMethodCache[name]
            generated_attribute_methods.module_eval { define_method name, method }
          end
        else
          def define_method_attribute(name)
            # ...
          end
        end
```